

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Turing Test:** An operational test for intelligent behavior, where a human evaluator engages in natural language conversations with both a human and a machine, attempting to distinguish them. The machine passes if the evaluator cannot reliably identify it as such.
- **Chinese Room Argument:** A thought experiment that challenges the validity of the Turing Test as a measure of true intelligence, arguing that a system can pass the test without possessing genuine understanding.
- **Minimax Search:** A decision-making algorithm used in game playing, where the algorithm explores the game tree to find the best move for the current player, assuming the opponent will also play optimally.
- **Alpha{-}Beta Pruning:** A search algorithm optimization technique that reduces the search space in Minimax search by eliminating branches that cannot affect the optimal decision.
- **Perceptron:** A fundamental unit in neural networks, consisting of weighted inputs, a summation node, and an activation function that produces an output based on the weighted sum of inputs and a threshold.
- **Reinforcement Learning:** A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties for its actions.
- **ImageNet Dataset:** A large database of images used for training and benchmarking computer vision models.
- **AlphaGo:** A computer program that achieved superhuman performance in the game of Go, combining deep learning and Monte Carlo tree search.
- **Matrix:** A rectangular array of numbers, symbols, or expressions, arranged in rows and columns.
- **Linear Transform:** A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c .
- **Matrix Multiplication:** An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of row i of the first matrix and column j of the second matrix. The inner dimensions must match.
- **Matrix Inverse:** For a square matrix A , its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition:** An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T .
- **Derivative:** A measure of how a function changes with respect to a change in its input.
- **Partial Derivative:** A derivative of a function of multiple variables with respect to one of those variables, holding the others constant.

- **Sigmoid Function:** A mathematical function that maps any input value to an output value between 0 and 1. Often used as an activation function in neural networks.
- **Inductive Reasoning:** The process of reasoning from specific observations to general conclusions.
- **Linear Regression:** A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Model:** A mathematical representation of a system or process, used to make predictions or understand relationships between variables.
- **Overfitting:** A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Decision Boundary:** A line or surface that separates different classes of data points in a classification problem.
- **0{-}1 Loss:** A loss function in binary classification that assigns a loss of 1 if the prediction is incorrect and 0 if it is correct.
- **Perceptron Loss:** A loss function in binary classification that is 0 if the prediction is correct and $-z$ otherwise, where z is the margin.
- **Hinge Loss:** A loss function in binary classification that is 0 if the margin is greater than or equal to 1 and $1 - z$ otherwise, where z is the margin.
- **Logistic Loss:** A loss function in binary classification that is $\log(1 + e^{\sup>-z</sup>})$, where z is the margin.
- **Gradient Descent:** An iterative optimization algorithm used to find the minimum of a function by repeatedly taking steps in the direction of the negative gradient.
- **Stochastic Gradient Descent:** A variant of gradient descent that updates the model parameters using the gradient calculated from a single randomly selected data point at each iteration.
- **Empirical Risk Minimizer:** A learning principle that aims to find the model that minimizes the average loss on the training data.

2 Problem Types

- **Understanding the Nature of Intelligence:** This problem explores the definition and characteristics of intelligence, both human and artificial. It examines various perspectives on what constitutes intelligence and how it can be measured or simulated.
- **Defining Artificial Intelligence:** This problem involves formulating a precise definition of artificial intelligence, considering different perspectives and historical contexts. The challenge lies in capturing the essence of AI while accounting for its evolving nature and diverse applications.
- **Passing the Turing Test:** This problem focuses on the implications of passing the Turing Test, a benchmark for machine intelligence. It explores whether passing the test truly indicates machine thinking or merely the ability to mimic human behavior.
- **Overcoming the Limitations of Search:** This problem addresses the computational challenges associated with exhaustive search in complex games like chess and Go. It explores the need for alternative approaches, such as machine learning, to overcome the limitations of brute-force search.
- **Addressing AI Safety Concerns:** This problem investigates the potential risks and ethical implications of AI systems, particularly concerning their vulnerability to adversarial attacks and the presence of biases in their decision-making processes.

- **Understanding the Differences Between Game Types:** This problem involves analyzing the differences between various game types, such as chess and soccer, in terms of their complexity, dynamics, and the challenges they pose for AI algorithms. It explores how these differences necessitate different approaches to game-playing AI.
- **Bridging the Gap Between Different AI Domains:** This problem explores the similarities and differences between seemingly disparate AI domains, such as game playing and speech synthesis, to identify common underlying principles and potential cross-domain applications of AI techniques.
- **Developing Robust and Ethical AI Systems:** This problem focuses on the development of AI systems that are not only effective but also safe, reliable, and free from biases. It explores the ethical considerations involved in designing and deploying AI systems in various real-world applications.
- **Explaining the Success of AI in Game Playing:** This problem examines the factors contributing to the success of AI in game playing, including the use of deep learning, search algorithms, and the combination of both. It explores the specific techniques employed in successful AI game-playing systems, such as AlphaGo.
- **Representing Linear Transformations with Matrices:** Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic:** The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations:** Systems of linear equations can be represented and solved using matrices and their inverses. This is crucial for various AI applications.
- **Matrix Transposition:** Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix. It's used in various AI algorithms for data manipulation.
- **Parallelization of Matrix Multiplication:** The slide discusses how matrix multiplication can be parallelized using domain decomposition to improve computational efficiency. This is important for handling large datasets in AI.
- **Calculus in AI:** The lecture will cover derivatives, partial derivatives, and finding maxima, minima, and gradients, which are essential for optimization in AI algorithms.
- **Linear Regression:** Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression:** Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting:** Fitting a curve to data points, potentially using polynomial functions of higher degrees.
- **Binary Classification:** Classifying data points into two categories (+1 or -1) using a linear model and a suitable loss function.
- **Overfitting:** Fitting a model too closely to the training data, resulting in poor generalization to unseen data.

3 Algorithm Solutions

- **Minimax Search:** A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.

- **Alpha{-}Beta Pruning:** An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.
- **Matrix Addition:** $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication:** $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication:** $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Transposition:** Swapping rows and columns of a matrix A to obtain A^T .
- **Solving Linear Equations with Invertible Matrices:** $Ax = b \implies x = A^{-1}b$
- **Derivative of Sigmoid Function:** $\frac{d}{dx}\sigma(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$
- **Multiple Linear Regression:** Find the optimal parameters β that minimize the sum of squared errors between the observed and predicted values. This is done by solving the equation $\beta = (X^T X)^{-1} X^T y$, where y is the vector of observed values, X is the design matrix of independent variables, and β is the vector of parameters to be estimated.
- **Perceptron Algorithm:** Iteratively update the weight vector w based on misclassified data points. If $\text{sign}(w^T x_n) \neq y_n$, then $w \leftarrow w + y_n x_n$.